Phytium飞腾

X100_NPU 部署深度学习模型实例

(以 YOLOv5s 为例)

（V1.1）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 05 月 15 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

目录

1 简介 3
1.1 文档简介 3
1.2 X100_NPU 简介 3
2 模型部署流程及运行所需软件栈 4
2.1 模型部署流程 4
2.2 模型转换和软件栈 4
3 YOLOv5s 目标检测模型准备 6
3.1 YOLOv5s 简介 6
3.2 YOLOv5s 模型准备 6
4 YOLOv5s 模型部署环境准备 8
4.1 检查硬件和驱动 8
4.2 获取开源 YOLOv5s 源码并安装开发环境 8
5 YOLOv5s 目标检测推理 9
5.1 基于 PhyNPUEngine 实现的 YOLOv5s 推理 10
5.1.1 加载依赖 10
5.1.2 YOLOv5s 推理实现 10
5.2 基于 PhyAIEngine 实现的 YOLOv5s 推理 11
5.2.1 依赖加载 11
5.2.2 YOLOv5s 推理实现 12
A 更新记录 13

1 简介

1.1 文档简介

本文档主要介绍了在飞腾 X100 套片的神经网络处理器(NPU)部署深度学习目标检测模型 YOLOv5s，包括模型转换、环境准备和推理开发三大部分，旨在为用户在 X100_NPU 上部署深度学习模型提供参考。

1.2 X100_NPU 简介

X100_NPU 是一种专门用于神经网络计算加速的部件，采用 ASIC 设计，具有较高的算力能耗比，算力 INT8 1TOPS，频率 300MHz，主要应用于图像领域神经网络推理，支持卷积、转置卷积、池化、全连、激活等 DNN 常用算子，支持 ONNX、TensorFlow、PyTorch、PaddlePaddle、TensorFlowLite、Caffe、Darknet 等框架模型转化，支持图像分类、目标检测、目标跟踪等常用模型，支持量化，计算精度 4~16Bits(可在性能和精度之间权衡)，功耗最大 2W，典型值约 1W，支持多种形式的低功耗模式。

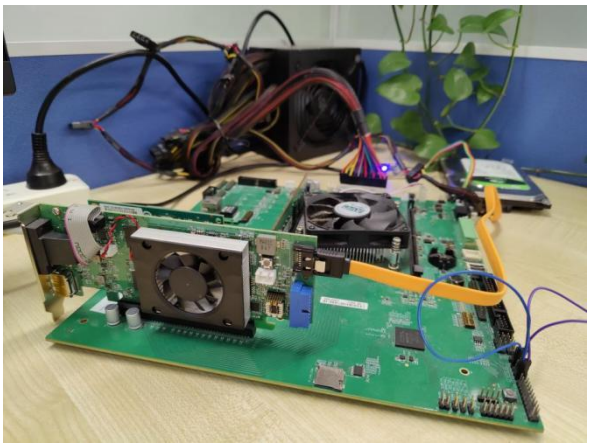

图 1-1 飞腾 D2000+X100 部署环境

2 模型部署流程及运行所需软件栈

2.1 模型部署流程

飞腾 X100_NPU 模型部署流程包括三个主要的过程：1-模型准备，2-环境准备，3-推理程序开发。模型部署流程如下图所示：

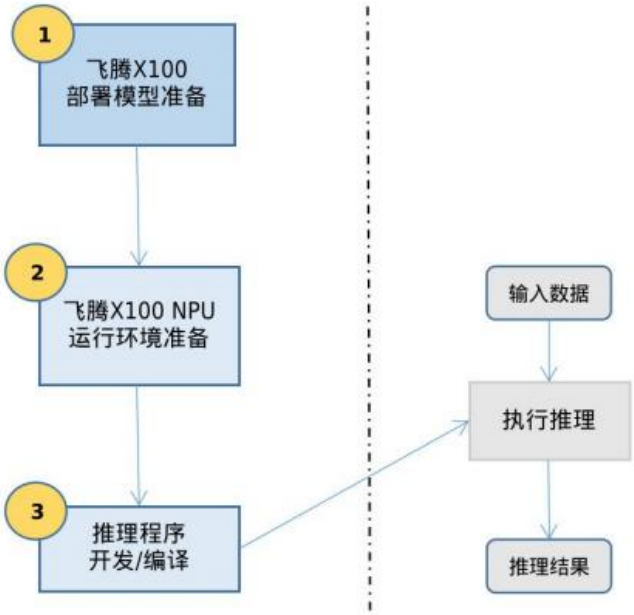

图 2-1 模型部署流程图

2.2 模型转换和软件栈

飞腾 X100_NPU 的使用需要基于如下模型转换以及相应的定制软件栈：

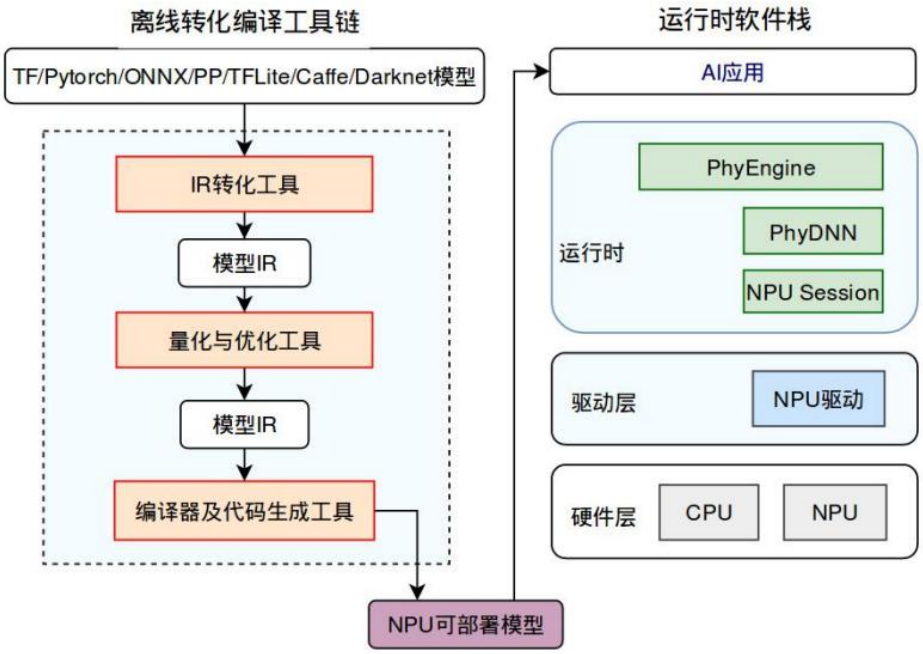

图 2-2 模型转换流程及软件栈

总体上，深度学习网络模型在飞腾 X100-NPU 上推理执行需要经过两个大的步骤：

(1) 基于 NCSDK(Neural Compute Software Development Kit)的模型转化编译，生成在 NPU 上可部署的模型文件(详见章 3-2)；

(2) 基于运行时软件栈的模型推理执行。

3 YOLOv5s 目标检测模型准备

3.1 YOLOv5s 简介

目标检测是计算机视觉的分支之一，该任务是在给定图像中识别特定物体并确定其位置。YOLO (you only look once) 是一种当下流行的目标检测模型之一，其能够对图像的物体进行识别和定位。YOLO 采用了多层卷积神经网络模型来完成端到端的物体检测，整个 YOLO 算法流程如下：首先将输入图像缩放到模型指定的输入大小，然后将其通过深度卷积神经网络进行处理，最终得出网络的预测结果，即检测到的物体的类别和位置信息。

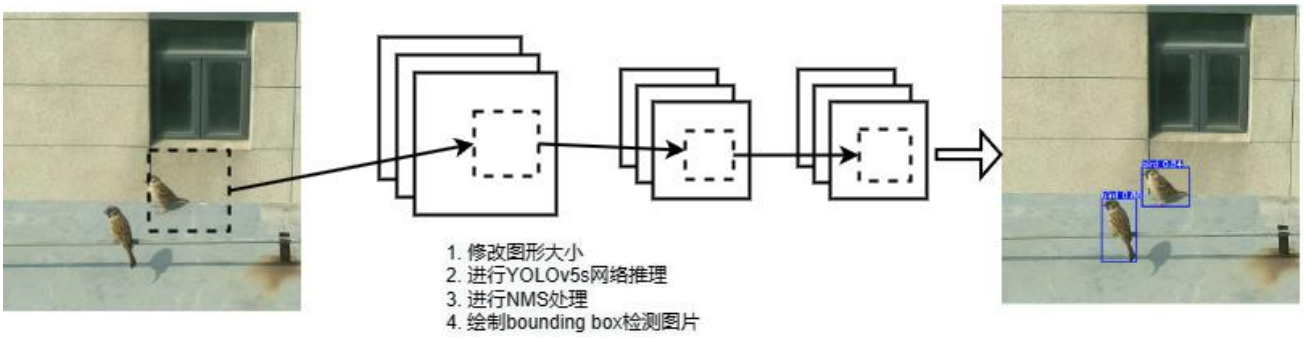

图 3-1 目标检测流程图

YOLOv5s：这是 YOLOv5 系列中最小的模型。“s”代表 “small”(小)。该模型在计算资源有限的设备上表现最佳，如移动设备或边缘设备。YOLOv5s 的检测速度最快，但准确度相对较低。YOLOv5 是一个开源的目标检测项目，源码网站地址为：https://github.com/ultralytics/yolov5。

3.2 YOLOv5s 模型准备

不同深度学习框架(如 ONNX、Tensorflow、Pytorch、paddlepaddle 等)下训练好的 YOLOv5s 模型具备对应框架下保留的格式，不能直接部署在 X100_npu 上，需要通过 NCSDK(Neural Compute Software Development Kit)工具用于将各类深度学习框架下的神经网络模型转化、编译成可以在飞腾硬件加速平台上运行的格式。

目前飞腾 NCSDK 转换工具支持 ONNX、TensorFlow、PyTorch、PaddlePaddle 、TensorFlow、LiteCaffe、Darknet 模型格式。

NCSDK 套件工具的核心是基于 relay IR 的计算图表示：

(1) 将可支持的开源深度学习框架生成的模型转成基于 relay IR 的模型。

(2) 基于 relay IR 进行计算图的优化、切分、调整以及直方图分析，输出仍为模型 IR。量化是基于设置的量化参数和数据样本进行。

优化后 relay IR 模型编译为 NPU 可部署模型，有两种途径编译：

(1) 直接基于 NPU 编译器。这种编译途径只适合整个模型都能在 NPU 上执行的情况。

(2) 基于 X100_build 编译器。这种编译途径不仅适合整个模型都能在 NPU 上执行的情况，还适合模型中有 NPU 不能运行的 layer 的情况。

转化编译之后就可以利用飞腾引擎部署到飞腾 NPU 上进行推理。NCSDK 转换编译流程详见下图。

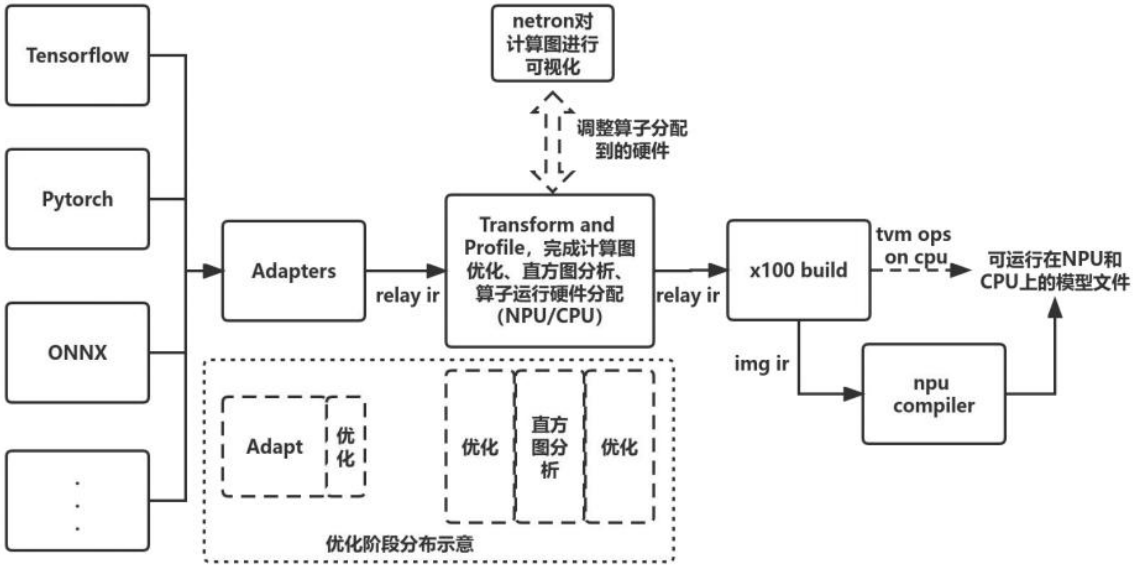

图 3-2 NCSDK 转化编译流程图

注：转化编译细节请详见《飞腾 NCSDK 使用手册》,如对 NCSDK 转换工具有需求请联系飞腾信息技术有限公司。

通过 NCSDK 中的编译工具可以生成两种类型的模型进行部署，对应关系如下表所示：

表 3-1 编译关系对应表

NCSDK 工具内编译器名称	未编译的模型类型	编译优化之后模型类型	推理引擎
NPU complier	YOLOv5s.onnx(以 ONNX 框架为例)	YOLOv5s.mbs.bin	phyNPUEngine 推理引擎
X100_build	YOLOv5s.onnx(以 ONNX 框架为例)	YOLOv5s.json YOLOv5s.tar YOLOv5s.so	PhyAIEngine推理引擎

		YOLOv5s.params	

4 YOLOv5s 模型部署环境准备

4.1 检查硬件和驱动

(1) 检查是否存在 NPU 设备或 NPU 是否开启：

```
$ lspci
    ......
    03:00.6 Processing accelerators: Device 1db7:dc24        #加载的 NPU 设备信息
    ......
```

(2) 检查是否存在 NPU 驱动：

```
$ lsmod | grep phy*
    Module              Size        Used by
    phy_npu             151552        0
    phy_mem_npu         110592        1   phy_npu
```

(3) 检查/dev/下是否有 NPU 设备描述符并赋予权限：

```
$ ls -al /dev/phy*
    crw------- 1 root root 10, May 11 10:53 phy_npu0     #phy_npu0 为 NPU 设备描述符

$ sudo chmod a+rw /dev/phy_npu0          #赋予权限
```

4.2 获取开源 YOLOv5s 源码并安装开发环境

(1) 执行下面的命令克隆 YOLOv5s 源码到本地：

```
$ git clone https://github.com/ultralytics/yolov5.git
```

(2) 安装 pip3，pip3 是 Python3 的包管理器工具，它可以方便地安装、升级和卸载 Python 包。

```
$ sudo apt-get install pip3
```

(3) 进入到 YOLOv5s 项目，里面有 requirements.txt 文件，可直接通过命令安装文件中提及的 Python 包。

```
$ pip3 install -r requirements.txt
```

5 YOLOv5s 目标检测推理

X100_NPU 模型部署运行基于 PhyEngine 推理引擎进行。推理程序工作流程如下图所示：

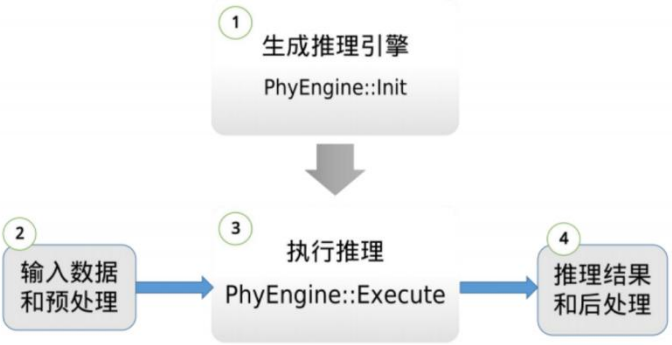

图 5-1 推理程序流程图

开发 YOLOv5s 推理程序只需要简单的 4 个步骤：

(1) 加载第三方库文件(表 5-1)，初始化和创建推理引擎(phyAIEngine 或者 phyNPUEngine)，通过调用 init(model_file)接口完成引擎的初始化；

(2) 准备输入数据，对输入的数据按 YOLOv5s 模型输入需要进行预处理；

(3) 输入数据推理执行，调用 execute()执行推理，返回执行结果；

(4) 推理结果后处理。

表 5-1 推理引擎与第三库关系表

推理引擎	模型类型	LIB 库
PhyNPUEngine 推理引擎	YOLOv5s.mbs.bin	__init__.py libnpusession.so libphycustom.so libnnasession.so _npu.so
PhyAIEngine 推理引擎	YOLOv5s.json YOLOv5s.tar YOLOv5s.so YOLOv5s.params	__init__.py libnpucompiler.so libnnasession.so libphyaiengine.so libPHYDNN.so libtvm_runtime.so pyphyaiengine.so

注：如对相关依赖库有需求，请联系飞腾信息技术有限公司。

5.1 基于 PhyNPUEngine 实现的 YOLOv5s 推理

5.1.1 加载依赖

在 YOLOv5s 项目的根目录下新建文件夹 phy_npu，将下述交付的依赖文件放到这个文件夹下：__init__.py、libnpusession.so、libphycustom.so、_npu.so。执行一下命令添加环境变量，之后就可正常加载：“import phy_npu”。

```
$ export LD_LIBRARY_PATH=/path/to/phy_npu
```

5.1.2 YOLOv5s 推理实现

在项目根目录的 models 文件夹存放编译生成的 YOLOv5s.mbs.bin 文件，并新建一个名为 YOLOv5s_PHY.py 文件，编写 YOLOv5s 推理的类的实现。只需将原有的 YOLOv5s 代码中的模型推理部分，替换为使用 NPU 推理即可。下面的代码是一个标准的使用 NPU 进行推理的类的实现。

```
import phy_npu

class YOLOv5s_npu(object):
    def __init__(self, mbs_path):
    session = phy_npu.createSession(0)
    self.network = session.network(mbs_path)
    in_keys = list(self.network.input.keys())
    out_keys = list(self.network.output.keys())
    print("name of input nodes: ", in_keys)  # 打印模型输入节点的名字
    print("name of output nodes: ", out_keys)  # 打印模型输出节点的名字
    self.inbufers = []
    self.outbufers = []
    self.outmems = []
    self.inmems = []
    for in_name in in_keys:
        self.inbufers.append(self.network.input[in_name])
        self.inmems.append(session.memory(self.inbufers[-1], in_name))
    for out_name in out_keys:
        self.outbufers.append(self.network.output[out_name])
        self.outmems.append(session.memory(self.outbufers[-1], out_name))

def __call__(self, inputs):
    for input, inmem in zip(inputs, self.inmems):
        inmem.from_float32_array(input)
```

```
    input_dict={inbufer.name:inmem for inbufer, inmem in zip(self.inbufers, self.inmems)}
    output_dict={outbufer.name: outmem for outbufer, outmem in zip(self.outbufers,
    self.outmems)}
    self.network.start(input_dict, output_dict)
    self.network.wait()
    return [x.to_float32_array() for x in self.outmems]
```

用户只需注意以下三点：

- 类名改成合适的名字；
- 根据打印的模型输入节点、模型输出节点的名字来调整输入张量、输出张量的顺序；
- 使用 model.mbs.bin 文件的路径初始化类。

修改 YOLOv5s 源码的 detect.py 文件，修改模型调用代码如下。注意的 model 的输入和输出都是 List，在对 input_tensors 中的元素赋值时，要参考打印的模型输入节点的名字顺序赋值，同样的，在取用 output_tensors 中的值时，也要参考打印的模型输出节点的名字的顺序正确取用。

```
model = YOLOv5s_npu(/path/to/YOLOv5s.mbs.bin)
output_tensors = model(input_tensors)
```

指定输入图片进行推理，运行 detect.py 文件。推理结果如下,推理耗时 0.163 秒：

```
image1/1/home/user/npu/distri_infer/infer_lib/tvm/python/example/yolov5/data/zidane.jpg:
640x640 2 persons, 2 ties, Done. (0.163s)
Speed: 6.2ms pre-process, 162.5ms inference, 187.0ms NMS per image at shape (1, 3,
640, 640)
```

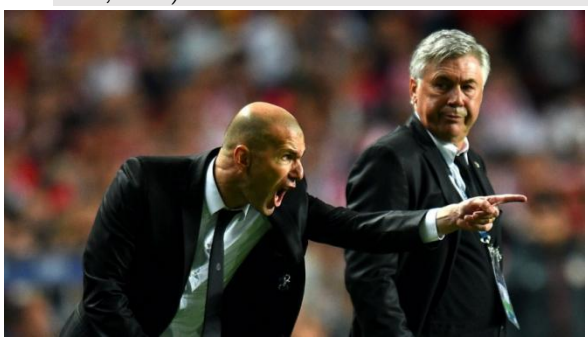

图 5-2 输入图片和 YOLOv5s 模型检测后的图片(PhyNPUEngine)

5.2 基于 PhyAIEngine 实现的 YOLOv5s 推理

5.2.1 依赖加载

在 YOLOv5s 项目的根目录下新建文件夹 phy_npu_tvm，将交付的依赖文件(表 5-1)放到这个文件夹下。执行一下命令添加环境变量，之后就可正常加载：“import phy_npu_tvm”。

```
$ export LD_LIBRARY_PATH=/path/to/phy_npu_tvm
```

5.2.2 YOLOv5s 推理实现

在项目根目录的 models 文件夹存放编译生成的 YOLOv5s.json、YOLOv5s.tar、YOLOv5s.so、YOLOv5s.params 文件，并新建一个名为 YOLOv5s_tvm.py 文件，编写 YOLOv5s 推理的类的实现。只需将原有的 YOLOv5s 代码中的模型推理部分，替换为使用 NPU 推理即可。下面的代码推理类的实现。

```
import phy_npu_tvm

class YOLOv5s_tvm(object):
    def __init__(self, model_path, in_dims, in_sizes, out_dims, out_sizes):
        self.executor = phy_npu_tvm.createExecutor(model_path)
        self.executor.set_ios(in_dims, in_sizes, out_dims, out_sizes)

    def __call__(self, inputs)
        self.executor.execute(inputs[0])
        res = self.executor.get_res()
        return res
```

用户只需注意以下三点：

- 类名改成合适的名字；
- 目前只支持单个输入张量、输出多个张量，对于多张量输入暂时不支持；
- 使用不带文件名后缀的模型文件的路径初始化类。

修改 YOLOv5s 源码的 detect.py 文件，修改模型调用代码如下：

```
Model_npu = YOLOv5s_tvm(“/path/to/YOLOv5s”,[4],[1,3,640,640],[3],[1,25200,85])
pred = model_npu([input.cpu().numpy()])[0]
```

指定输入图片进行推理，运行 detect.py 文件。推理结果如下，推理耗时 0.144 秒：

```
image1/1/home/user/npu/distri_infer/infer_lib/tvm/python/example/yolov5/data/images/messi.jpeg: 640x640 4 persons, 1 sports ball, Done. (0.144s)
Speed: 7.9ms pre-process, 144.4ms inference, 13.1ms NMS per image at shape (1, 3, 640, 640)
```

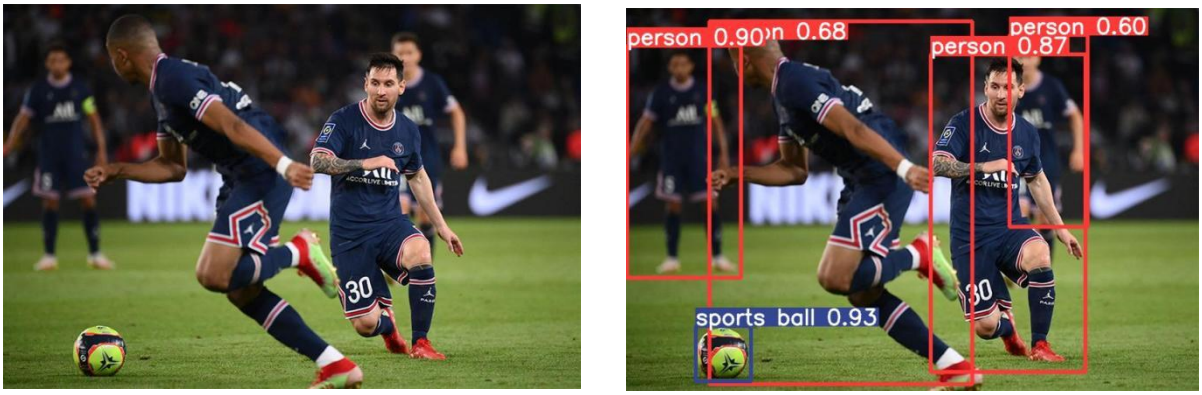

图 5-3 输入图片和 YOLOv5s 模型检测后的图片(PhyAIEngine)

A 更新记录

发布日期	作者	说明
2023-05-15	刘才诚	初稿(v1.0)
2023-06-05	刘才诚	第一次修改(v1.1)：调整图片，修改 YOLO 模型说明